Experimental Report

Stochastic Gradient Descent for Neural Networks using MPI

Course: DSA5208 Scalable Distributed Computing in Data Science

Group Members:

- Name: Zhao Yuchen Student ID: A0333547B
- Name: Liang Ran Student ID: A0328124L

1. Introduction

This report details the implementation and experimental results of a parallel Stochastic Gradient Descent algorithm for training a single-hidden-layer neural network. The project's core objective was to leverage the Message Passing Interface to distribute the computational workload across multiple processes, thereby accelerating the training process on large datasets. The algorithm was applied to the *NYC Taxi Fare dataset* to build a predictive model for taxi fare amounts.

The primary goals are as follows:

- To implement an SGD algorithm where the dataset is partitioned across MPI processes.
- To ensure the implementation works efficiently for any number of processes without centralizing the entire dataset on any single process.
- To evaluate the model's performance using different activation functions and batch sizes.
- To analyze the scalability of the implementation by measuring training times with varying numbers of processes.

2. Dataset and Preprocessing

2.1. Dataset Description

The dataset used in this experiment is the *NYC Taxi Fare Dataset* sourced from the Kaggle platform. The original data file, *nytaxi2022.csv*, is approximately more than 3.0 GB in size and contains over 39 million records of taxi trips in New York City for the year 2022.

The dataset contains rich trip information features. Based on experimental requirements, we selected the following features as model inputs to build a machine learning model to predict the total fare amount(column *total_amount*):

Variable Name	Variable Type	Description
tpep_pickup_datetime	object (timestamp)	The date and time when the passenger was picked up.
tpep_dropoff_datetime	object (timestamp)	The date and time when the passenger was dropped off.
passenger_count	float64	The number of passengers in the taxi.
trip_distance	float64	The total distance of the trip in miles or kilometers.
RatecodeID	float64	The rate code assigned to the trip, representing fare types.
PULocationID	int64	The unique identifier for the pickup location.
DOLocationID	int64	The unique identifier for the drop-off location.
payment_type	int64	The method of payment used by the passenger.
extra	float64	Additional charges applied during the trip.

2.2. Data Preprocessing[1]

2.2.1. Missing Values

We identified that approximately 1.3 million rows had missing values in both of *passenger_count* and *RatecodeID* columns simultaneously. This accounted for about 3.4% of the initial total samples. Given that this proportion is relatively small and the missingness in these critical features is difficult to impute accurately and reasonably, we decided to **remove all rows containing these missing values** to maintain data rigor and integrity.

2.2.2. Data Transformation and Feature Engineering

- Timestamp Data

The original *tpep_pickup_datetime* and *tpep_dropoff_datetime* were stored as strings in the 'MM/DD/YYYY HH:MM:SS AM/PM' format. We called the PARSE_TIMESTAMP function to convert them into standard TIMESTAMP types, which is a prerequisite for any temporal calculations.Utilizing the parsed timestamps, we created a new feature: ***trip_duration_minutes*** to calculate the difference between the drop off times and pick up times using the TIMESTAMP_DIFF function.

- Categorical Data

Categorical features such as *RatecodeID, payment_type, PULocationID,* and *DOLocationID* represents distinct categories. We employed different encoding strategies based on their cardinality:

Low-Cardinality Categorical Variables (**One-Hot Encoding**): For *RatecodeID* and *payment_type*, which have relatively low cardinality, we used One-Hot Encoding and created new binary feature columns for each category (e.g., RatecodeID_1, RatecodeID_2, ..., payment_type_1, ...). This directly converts categories into a sparse numerical format suitable for neural network input.

High-Cardinality Categorical Variables (**Reserved for Embedding**): For *PULocationID* and *DOLocationID*, which have very high cardinality (over 200 categories), using One-Hot Encoding would lead to a dimensionality explosion, creating overly sparse input. Therefore, at this stage, we only retained their original ID values.

- Numerical Data

Firstly, we focused on the objective variable *total_amount*, and noticed some existing negative values. To simplify the problem, we just kept the **non-negative records**. To mitigate the training instability caused by the skewed distribution, we applied a log transformation to the target variable, creating a new feature ***total_amount_log*** using LOG(1 + total_amount). This transformation converts an exponential distribution to an approximately normal distribution, improving the optimization process of gradient descent.

Then, for other numerical features, such as *trip_distance, passenger_count, extra,* and

[1] Code is provided in the appendix. Tools: Google Cloud — BigQuery.

trip_duration_minutes, we applied **Z-score standardization** to ensure stable and efficient gradient descent optimization.

2.2.3. Dataset Split

After completing all data preprocessing and feature engineering, the final dataset was split into a training set and a test set using a 7:3 ratio. We employed a **deterministic hash-based splitting method**, using the *tpep_pickup_datetime* column as the key. Finally, the rows counts of training set and test set are around 26.6 million and 11.4 million respectively.

3. Methodology and Implementation

3.1. Parallelization Strategy

3.1.1. Data Sharding

We shard the parquet dataset across MPI ranks to achieve parallel I/O and balanced compute. In the training path, rank 0 glob-lists all **.parquet*, derives feature columns, then splits file paths into size shards and scatters them to workers; feature/target meta are broadcast so every rank builds data with a consistent schema. Each rank loads only its own files and proceeds independently. During evaluation, we similarly list test files on rank 0, split into size parts, and scatter to ranks. This ensures no duplicate reads and a balanced workload. For the streaming trainer with risk curves, sharding is implemented by simple striding files [rank::WORLD], and each rank samples random row-groups from its shard at each step, avoiding full concatenation and reducing memory pressure.

3.1.2. MPI Parallelization

We use synchronous data-parallel SGD. Each iteration, every rank computes local gradients on its mini-batch, then performs MPI All-reduce to average gradients; the averaged grads are (optionally) globally clipped for stability and applied to update parameters—keeping weights synchronized across processes.

Feature-wise normalization statistics and (when using log targets) target mean/std are also aggregated via MPI to ensure consistent scaling. In the curve-logging trainer, a parallel risk estimate is computed by all-reducing local SSE and sample counts. During evaluation, per-rank SSE is all-reduced to report global RMSE.

3.1.3. Arbitrary Process Number Design

We allow users to choose the MPI process count (NP) interactively while safeguarding correctness and reproducibility. At launch, the grid script prompts for NP (default 4) and validates the input as a non-negative integer within the allowed range. Invalid or out-of-range values terminate early with a clear error code and message, preventing ambiguous runs.

To avoid over-subscription, the script probes available local slots (logical CPUs / hardware

threads) and sanitizes the result to be ≥1. It also pins intra-process thread pools to one, ensuring that each MPI rank does not silently spawn additional compute threads.

If the requested NP exceeds detected slots, the script does not fail blindly; instead it offers an explicit, auditable choice: (i) reduce NP to the slot count (recommended), (ii) treat hardware threads as schedulable (--use-hwthread-cpus), (iii) permit oversubscription (--oversubscribe, potentially slower), or (iv) cancel. The selected option is incorporated into a unified mpirun prefix so that every training and evaluation command is launched with the same, clearly logged scheduling configuration.

3.2. Model Architecture

3.2.1. Single Hidden Layer Neural Network and Activation

We adopt a single-hidden-layer MLP for scalar regression. Parameters are $W_1 \in \mathbb{R}^{H\times D}, b_1 \in \mathbb{R}^{H\times 1},\ W_2 \in \mathbb{R}^{1\times H}, b_2 \in \mathbb{R}^{1\times 1}$. The forward pass is:

$$Z_1 = W_1 X + b_1,\ \ A_1 = \phi(Z_1),\ \ \hat{y} = W_2 A_1 + b_2.$$

We use mean-squared loss per mini-batch. Backpropagation computes closed-form gradients for all parameters, and updates are applied synchronously across ranks after MPI Allreduce averaging.

We support ReLU, Tanh, and Sigmoid. The training script registers each activation together with its derivative for backprop; the evaluation path reuses the forward activations.

3.2.2. Parameters

Core hyper-parameters include hidden size, activation, learning rate, batch size, epochs , and optional global gradient clip. The details are as follows:

Parameter	Description	Values
ACTS	Activation functions	("relu" "tanh" "sigmoid")
BSS	Training batch sizes swept in the grid	(256 512 1024 2048 4096)
NP	MPI process count	4
HIDDEN	Hidden layer width	128
EPOCHS	Training epochs	4
LR	Learning rate	1e-3
--clip	Gradient clipping (L2)	5
--max_steps_per_epoch	Cap steps per epoch	200

3.3. Training Procedure

3.3.1. Optimizer

We use synchronous data-parallel SGD. Each iteration, every rank computes local minibatch gradients; then an MPI Allreduce sums them and we divide by WORLD/size to obtain the averaged gradient, which is applied to the shared parameters.

Optional global L2 gradient clipping is applied after averaging to stabilize updates. In *mpi_train_save.py*, minibatches are formed by shuffling an index permutation each epoch; in *mpi_train_with_curve.py*, a Parquet minibatcher samples random row-groups per step to stream data without concatenating the whole shard. The training loop logs progress every few steps; the curve trainer additionally computes a parallel risk estimate each curve_every steps via Allreduce and appends it to a CSV when enabled.

3.3.2. Stopping Condition

Training is epoch-bounded and step-bounded: the outer loop runs for epochs; inside each epoch, we stop either when all batches in the current epoch are consumed or when max_steps_per_epoch is reached whichever comes first. There is no early-stopping criterion in the trainers; runs terminate after saving a checkpoint and printing total train time on rank 0.

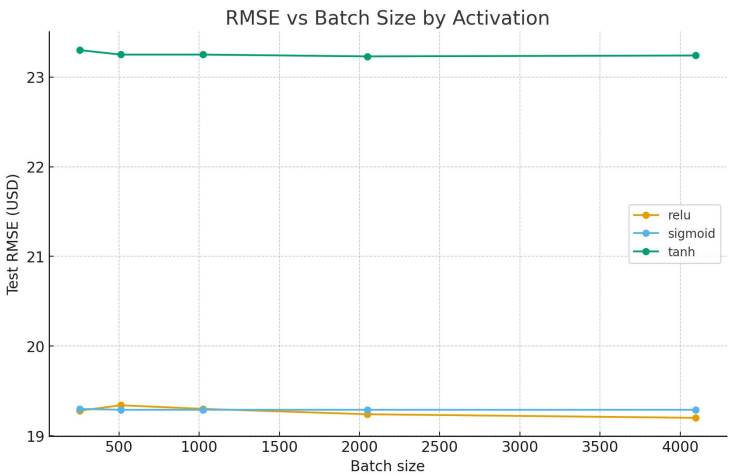
4. Experimental Results

4.1. Training Logs and RMSE Curves

4.1.1. Activation and batch size comparison

Unless otherwise noted: hidden=128, epochs=4, lr=1e-3, max_steps_per_epoch=200. We swept activation ∈ {ReLU, Sigmoid, Tanh} and batch ∈ {256, 512, 1024, 2048, 4096}. Evaluation used the same test split across runs.

The test RMSE is shown in the charts below:



For activation functions:

- ReLU: Best RMSE ≈ 19.20 USD at batch=4096; strong accuracy, but training time is noticeably longer at this batch size.
- Sigmoid: Best RMSE ≈ 19.29 USD at batch=512; accuracy is essentially on par with ReLU while being much faster to train.
- Tanh: Best RMSE ≈ 23.23 USD at batch=2048; clearly worse than ReLU/Sigmoid under this budget.

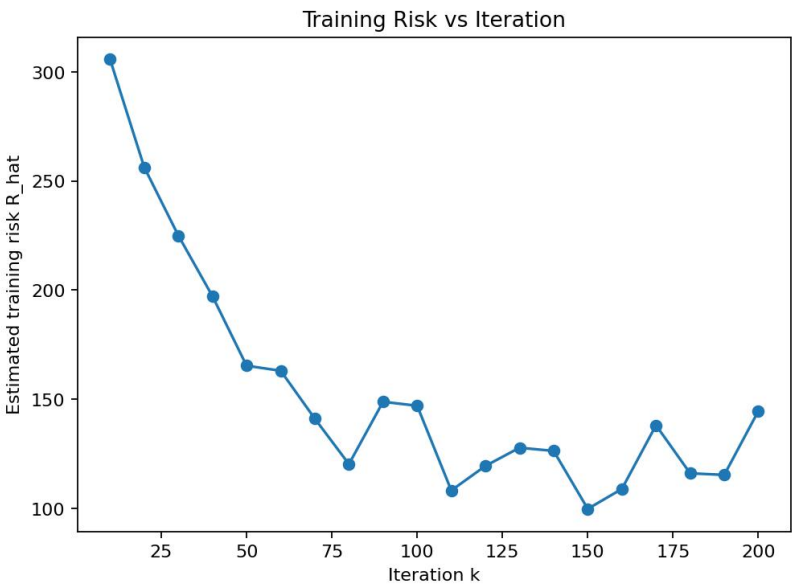
Conclusion:

- When accuracy is the priority, ReLU@4096 is the absolute best but only marginally ahead of Sigmoid.
- Under this setting, Tanh is not competitive; it can be de-prioritized in future sweeps.

4.1.2. Convergence curves

Main parameters:

- Hidden size --hidden: 128
- Learning rate --lr: 1e-3
- Batch size --batch: 1024
- Activations: ReLU
- --epochs: 4; --max_steps_per_epoch: 200
- Curve logging --curve_every: every 20 steps by default



The chart above shows the estimated training risk $\hat{R}$ against iteration k. The curve shows a steep initial decline followed by a shallower descent that settles into a stable plateau with small stochastic fluctuations.

This pattern indicates that (i) the learning rate and optimizer quickly move the parameters into a good basin, and (ii) training proceeds stably thereafter without signs of divergence. At each SGD update, every MPI rank computes its local sum of squared errors (SSE) on the current mini-batch. An MPI Allreduce then aggregates the global SSE and the global sample count to form

$$\widehat{\mathrm{R}} = \frac{1}{2}\frac{\mathrm{SSE}_{\mathrm{global}}}{\mathrm{M}_{\mathrm{global}}}$$

Gradients are also Allreduced and averaged across ranks before a synchronized parameter update; global L2 gradient clipping is applied to enhance stability. The trainer logs $\widehat{\mathrm{R}}$ at a fixed interval , while epochs run with a fixed number of steps.

4.2. Training Time Analysis

This analysis is a sub-experiment within our training study. We vary the MPI process count np ∈ {1, 2, 4, 8} on a single machine of type c2-standard-8 (8 vCPUs, 32 GB RAM).

The training times are as follows:

np	activation	batch	hidden	epochs	lr	train_time_s	flags
1	relu	1024	128	2	1e-3	2.9	
2	relu	1024	128	2	1e-3	1.9	
4	relu	1024	128	2	1e-3	1.4	
8	relu	1024	128	2	1e-3	2.1	use-hwthread-cpus

We train the same one-hidden-layer MLP for regression under identical data and hyperparameters: activation ReLU, hidden size 128, batch size 1024, 2 epochs, learning rate 1e-3. Wall-clock training time is measured end-to-end. For np=8, the run used --use-hwthread-cpus (ranks mapped to hyper-threads).

Observed results (from the timing table).

- np=1: 2.9 s (baseline, speedup 1.00×, efficiency 1.00)
- np=2: 1.9 s → 1.53× speedup vs 1-proc, 76% efficiency
- np=4: 1.4 s → 2.07× speedup vs 1-proc, 52% efficiency
- np=8: 2.1 s → 1.38× speedup vs 1-proc, 17% efficiency, and slower than np=4

We observe the classic “speedup then diminishing returns” pattern. Gains are clear up to np=4, but performance regresses at np=8. Primary factors may as follows:

- Collective-communication overhead (e.g., gradient Allreduce) rises with rank count.
- The workload is small (2 epochs), so fixed costs dominate at higher np.
- With hyper-thread mapping at np=8, ranks share physical cores, causing execution-unit/cache contention and higher memory-bandwidth pressure.
- More ranks amplify I/O/cache pressure and synchronization stalls.

Appendix(code in google cloud--BigQuery)

01: time_duration_calculation

```
CREATE OR REPLACE TABLE `data-preprocess-472706.nytaxi_dataset.taxi_data_processed` AS

SELECT
  -- considering AM/PM format
  PARSE_TIMESTAMP('%m/%d/%Y %I:%M:%S %p', tpep_pickup_datetime) AS tpep_pickup_datetime,
  PARSE_TIMESTAMP('%m/%d/%Y %I:%M:%S %p', tpep_dropoff_datetime) AS tpep_dropoff_datetime,

  -- compute trip duration(min)
  TIMESTAMP_DIFF(
    PARSE_TIMESTAMP('%m/%d/%Y %I:%M:%S %p', tpep_dropoff_datetime),
    PARSE_TIMESTAMP('%m/%d/%Y %I:%M:%S %p', tpep_pickup_datetime),
    MINUTE
  ) AS trip_duration_minutes,

  -- keep others
  passenger_count,
  trip_distance,
  RatecodeID,
  PULocationID,
  DOLocationID,
  payment_type,
  extra,
  total_amount

FROM
  `data-preprocess-472706.nytaxi_dataset.nytaxi2022_clean`

-- outlier detect
WHERE
  tpep_pickup_datetime IS NOT NULL
  AND tpep_dropoff_datetime IS NOT NULL;
```

02: one-hot & standardization

```
-- create final table with standardization, encoding, and log-transformed target
CREATE OR REPLACE TABLE `data-preprocess-472706.nytaxi_dataset.taxi_data_final` AS
WITH stats AS (
  -- precalculate for standardization
```

```sql
  SELECT
    AVG(trip_distance) AS mean_trip_distance,
    STDDEV(trip_distance) AS std_trip_distance,
    AVG(passenger_count) AS mean_passenger_count,
    STDDEV(passenger_count) AS std_passenger_count,
    AVG(extra) AS mean_extra,
    STDDEV(extra) AS std_extra,
    AVG(trip_duration_minutes) AS mean_trip_duration,
    STDDEV(trip_duration_minutes) AS std_trip_duration
  FROM `data-preprocess-472706.nytaxi_dataset.taxi_data_processed`
),
encoded AS (
  SELECT
    -- keep timestamp
    tpep_pickup_datetime,
    tpep_dropoff_datetime,

    -- One-hot: RatecodeID
    CASE WHEN RatecodeID = 1 THEN 1 ELSE 0 END AS RatecodeID_1,
    CASE WHEN RatecodeID = 2 THEN 1 ELSE 0 END AS RatecodeID_2,
    CASE WHEN RatecodeID = 3 THEN 1 ELSE 0 END AS RatecodeID_3,
    CASE WHEN RatecodeID = 4 THEN 1 ELSE 0 END AS RatecodeID_4,
    CASE WHEN RatecodeID = 5 THEN 1 ELSE 0 END AS RatecodeID_5,
    CASE WHEN RatecodeID = 6 THEN 1 ELSE 0 END AS RatecodeID_6,
    CASE WHEN RatecodeID = 99 THEN 1 ELSE 0 END AS RatecodeID_99,

    -- One-hot: payment_type
    CASE WHEN payment_type = 1 THEN 1 ELSE 0 END AS payment_type_1,
    CASE WHEN payment_type = 2 THEN 1 ELSE 0 END AS payment_type_2,
    CASE WHEN payment_type = 3 THEN 1 ELSE 0 END AS payment_type_3,
    CASE WHEN payment_type = 4 THEN 1 ELSE 0 END AS payment_type_4,
    CASE WHEN payment_type = 5 THEN 1 ELSE 0 END AS payment_type_5,

    -- ID maintain: for coming embedding
    PULocationID,
    DOLocationID,

    -- original numerical features(for join stats)
    trip_distance,
    passenger_count,
    extra,
    trip_duration_minutes,

    -- objective
```

```sql
    total_amount
  FROM `data-preprocess-472706.nytaxi_dataset.taxi_data_processed`
  WHERE total_amount > 0
)
SELECT
  e.tpep_pickup_datetime,
  e.tpep_dropoff_datetime,

  -- One-hot features
  e.RatecodeID_1, e.RatecodeID_2, e.RatecodeID_3, e.RatecodeID_4,
  e.RatecodeID_5, e.RatecodeID_6, e.RatecodeID_99,

  e.payment_type_1, e.payment_type_2, e.payment_type_3, e.payment_type_4,
e.payment_type_5,

  -- Embedding IDs
  e.PULocationID,
  e.DOLocationID,

  -- standardization
  SAFE_DIVIDE(e.trip_distance - s.mean_trip_distance, s.std_trip_distance) AS
trip_distance_std,
  SAFE_DIVIDE(e.passenger_count - s.mean_passenger_count, s.std_passenger_count) AS
passenger_count_std,
  SAFE_DIVIDE(e.extra - s.mean_extra, s.std_extra) AS extra_std,
  SAFE_DIVIDE(e.trip_duration_minutes - s.mean_trip_duration, s.std_trip_duration) AS
trip_duration_std,

  -- objective:
  e.total_amount,
  LOG(1 + e.total_amount) AS total_amount_log
FROM encoded e
CROSS JOIN stats s;
```

03: data_split

```sql
-- train (70%)
CREATE OR REPLACE TABLE `data-preprocess-472706.nytaxi_dataset.taxi_train` AS
SELECT *
FROM `data-preprocess-472706.nytaxi_dataset.taxi_data_final`
WHERE MOD(ABS(FARM_FINGERPRINT(CAST(tpep_pickup_datetime AS STRING))), 10) < 7;

-- test (30%)
CREATE OR REPLACE TABLE `data-preprocess-472706.nytaxi_dataset.taxi_test` AS
```

```sql
SELECT *
FROM `data-preprocess-472706.nytaxi_dataset.taxi_data_final`
WHERE MOD(ABS(FARM_FINGERPRINT(CAST(tpep_pickup_datetime AS STRING))), 10) >= 7;
```